Intrusion Detection on UNSW-NB15

Binary and Multiclass Classification of Network Traffic

Simone Montagnini

University of Bologna

September 2026

Outline

1. Introduction
2. Exploratory Data Analysis
3. Preprocessing
4. Attribute Selection
5. Classification
6. Conclusions

Project Objectives

This project addresses intrusion detection on network traffic through two classification tasks:

- **Binary classification**: distinction between normal and malicious traffic (`label`)
- **Multiclass classification**: identification of the specific threat among 9 possible types (`attack_cat`)

Source

Kaggle competition: *Advanced AI for Cybersecurity — Intrusion Detection with UNSW-NB15*

The Dataset — UNSW-NB15

- Realistic network traffic dataset with both normal and malicious network activity
- **175,341** training records, **36** original features (categorical and numerical)
- Two target columns:
  - `label`: 0 = normal, 1 = attack
  - `attack_cat`: Normal, Generic, Exploits, Fuzzers, DoS, Reconnaissance, Analysis, Backdoor, Shellcode, Worms

Target Distribution

Binary (`label`)

- Attack: 68.1%
- Normal: 31.9%

Multiclass (`attack_cat`)

- Normal: 56,000
- Generic: 40,000
- Exploits: 33,393
- Fuzzers: 18,184
- DoS: 12,264
- Reconnaissance: 10,491
- Analysis: 2,000
- Backdoor: 1,746
- Shellcode: 1,133
- **Worms: 130**

Balancing

The binary target is reasonably balanced, but `attack_cat` is severely imbalanced: Worms has 430× fewer examples than Normal — need for **class-weighting** strategies in the multiclass model.

Data Quality

- **Duplicate rows**: 78,519 out of 175,341 (**44.8%**)
  - Removed before splitting, to avoid data leakage between train and test
  - Distribution shift: the binary target becomes nearly balanced (**50.5% Normal / 49.5% Attack**)
- **Categorical features profile**:
  - `proto` (High cardinality): 133 unique values (`tcp`/`udp` $\approx$ 82% of traffic)
  - `service`: 13 values, 54% missing/unknown (-)
  - `state`: 9 values

Skewed Numerical Features

Several traffic features show extreme right-skewed distributions:

Feature	**Skewness**
trans_depth	167.3
response_body_len	76.3
sbytes	45.3
sloss	44.8
dloss	41.4
spkts	40.2

29 features exceeded the skewness threshold (> 1.0) during raw EDA; after deduplication on the train split, **27 features** remained skewed and were transformed via **log1p**.

Data Preparation & Transformation

1. **Duplicate removal**: removed 78,519 duplicate rows before splitting
2. **Train/test split**: 67/33, stratified on `attack_cat` to preserve class proportions across both tasks
3. **Encoding**: `OneHotEncoder` on `proto`, `service`, `state` — 184 total features
4. **Skewness correction**: `log1p` transform on 27 features (skewness > 1.0)

Avoiding Data Leakage

Key methodological choice

Preprocessing steps are **fit exclusively on the training set** and mapped to the test set via `transform()` in order to prevent data leakage.

- **Strict separation**: Fitting encoders on the entire dataset injects future knowledge into the training phase
- **Handling unseen data**: Rare categories (e.g. within the 133 `proto` values) might appear exclusively in the test set
- **Fault tolerance**: `OneHotEncoder` is configured with `handle_unknown='ignore'` to safely bypass novel categories without causing execution failures

SelectKBest

- **Supervised feature selection**: each feature is scored against the target, keeping only the top k features
- **Score function**: ANOVA F-value (`f_classif`)
  - `mutual_info_classif` was excluded due to extreme computation times on this dataset's dimensionality.
- **Selection**: $k = 50$ out of 184 features; it's embedded inside a `Pipeline` with $k \in \{30, 40, 50\}$ tuned in Cross-Validation

Interpretability Preservation

SelectKBest retains the original traffic attributes. This ensures the model remains transparent, enabling direct attribution of malicious activity to specific network signals.

PCA

- **Prerequisite scaling**: features standardized (`StandardScaler`), then PCA fitted on the training set only
- **Explained variance**: retaining 90% of the variance requires **136 components** (118 for 80%)
- **Limited reduction**: caused by the numerous one-hot encoded columns: once standardized, these sparse binary features are nearly uncorrelated with each other and cannot be effectively compressed

Component Analysis & Feature Contribution

Rather than being dominated by the numerous one-hot encoded columns `proto`, the principal components exhibit a mix of underlying data types. For instance, PC2 is driven almost entirely by timing and byte-related numerical features (`sinpkt`, `dload`, `tcprtt`)

Feature Selection Comparison

	SelectKBest	**PCA**
Dimensionality	30–50 features	136 components
Interpretability	High (original domain)	Low (linear combinations)
Transformation	Supervised (target-aware)	Unsupervised
Training efficiency	Acceptable	Impractical

Design Choice

PCA was excluded from the classification pipeline: even with a reduced number of components, the computational overhead during GridSearchCV was impractical. Classification proceeds with SelectKBest only.

Classification Methodology

- **Pipeline**: Decision Tree and Random Forest wrapped in a `Pipeline` (`VarianceThreshold` + `SelectKBest` + `Classifier`), compared against an **all-feature (184) Random Forest baseline**
- **Hyperparameter Tuning**: textttselector__k $(30, 40, 50)$, `max_depth` and `criterion` (DT), `n_estimators` and `max_depth` (RF)
- **Imbalance Handling**: Included `class_weight` (`None` vs `'balanced'`) to evaluate the best strategy for rare threat vectors
- **Validation**: 3-fold `StratifiedKFold` on training data; test set held out for final evaluation only

Evaluation Metrics

Accuracy is misleading on highly imbalanced datasets. Models are evaluated using **F1-Score** (Binary classification) and **Macro F1-Score** (Multiclass classification) to weigh the performance on minority classes equally.

Binary Classification Results

Cross-Validation Performance

Metric	RF Base	DT	RF Best
Accuracy	–	0.913	0.919
Precision	–	0.879	0.885
Recall	–	0.956	0.963
F1-Score	**0.919**	**0.916**	**0.922**

- **Test Set (RF, $k = 50$): F1 = 0.921**, Attack Recall **97%**, False Positive Rate **13%** on normal traffic.
- **Class Weight Effect**: at fixed optimal parameters, `None` (0.922) and `'balanced'` (0.921) perform identically (target is already balanced after deduplication).

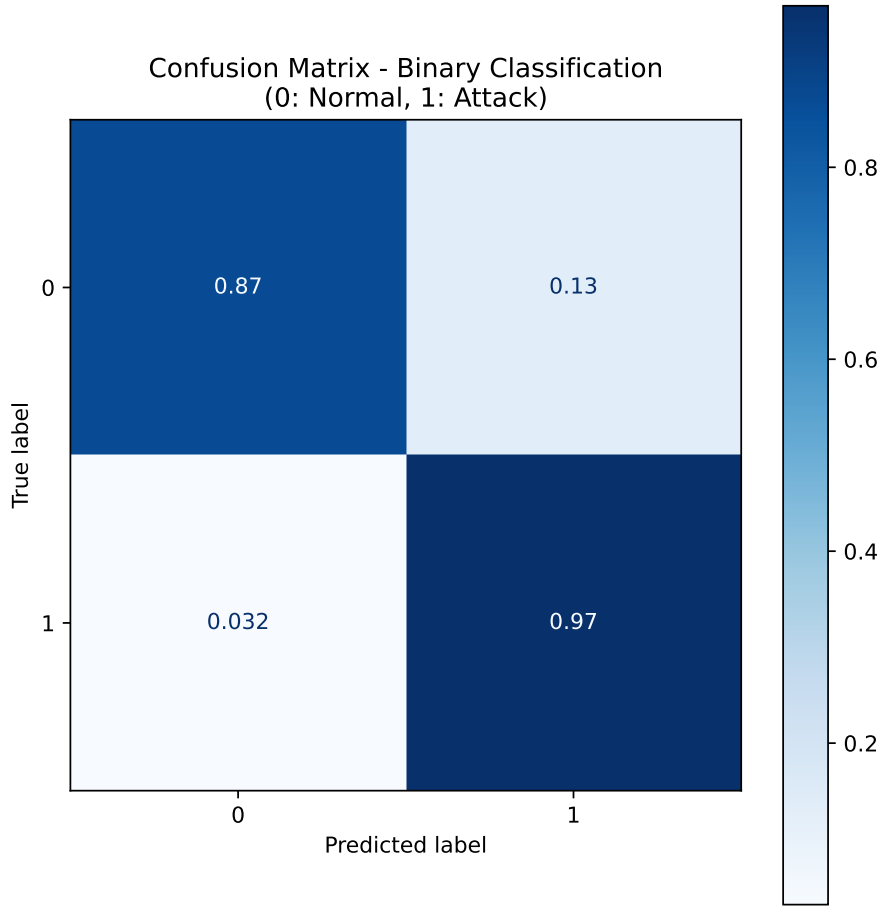


Figure: Binary Confusion Matrix

Multiclass Classification Results

The Multiclass Challenge

- **Random Forest ($k = 40$)**: achieves a **Macro F1 of 0.561** on the test set (**0.559** in CV vs **0.540** for DT and **0.461** for the 184-feature Baseline)
- **Majority classes** (Normal, Exploits) are predicted with high confidence
- **Minority classes**: low precision due to severe overlap and scarcity (Analysis 0.17, Backdoor 0.16, Shellcode 0.26)

Impact of Balancing

Thanks to `class_weight='balanced'`, the model is also able to detect instances of ultra-rare attacks (like Worms), preventing them from being completely ignored.

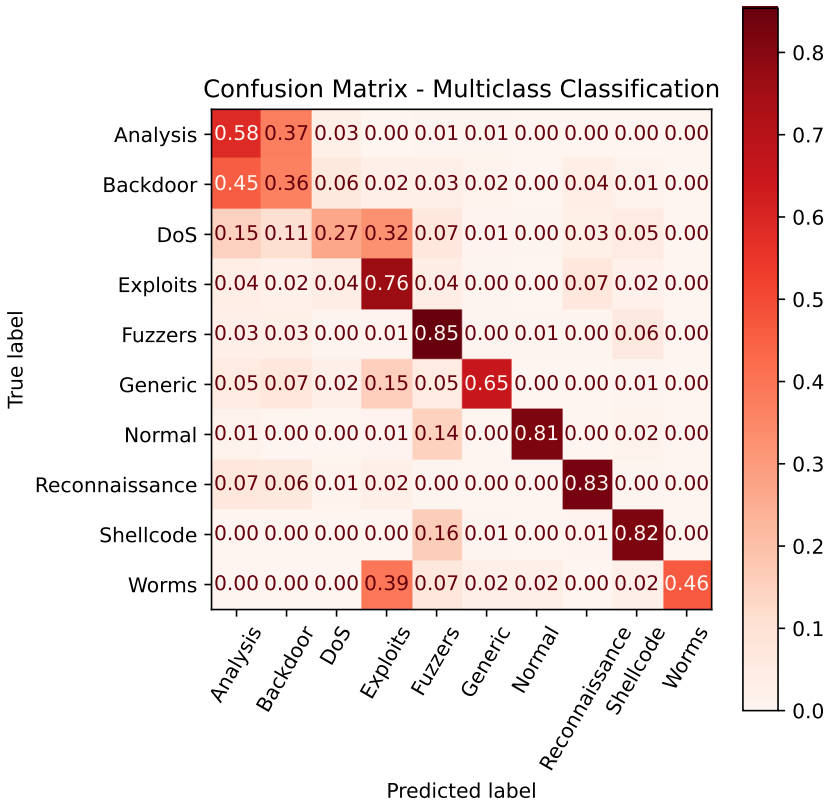
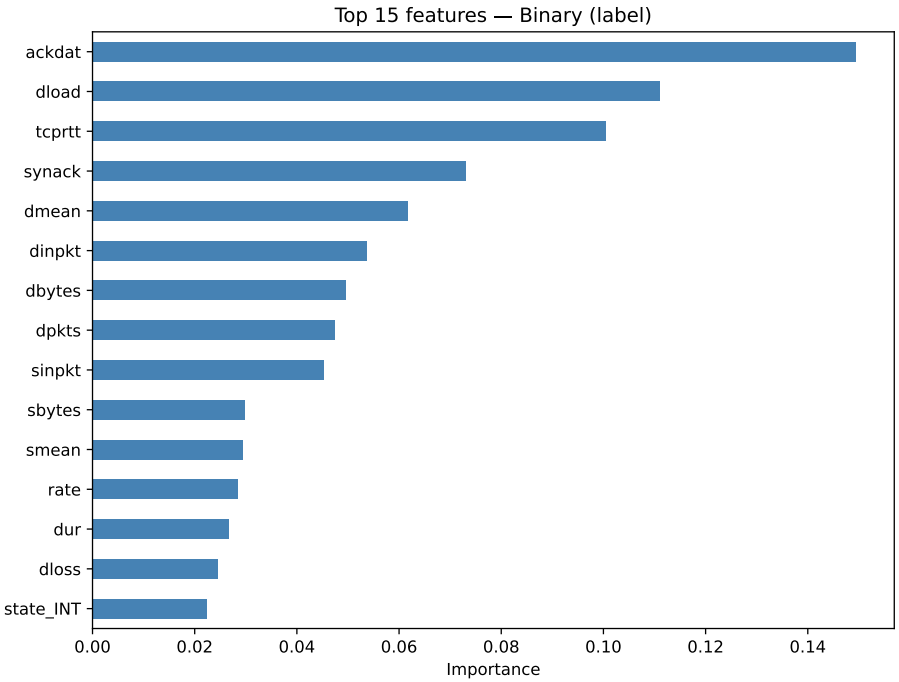
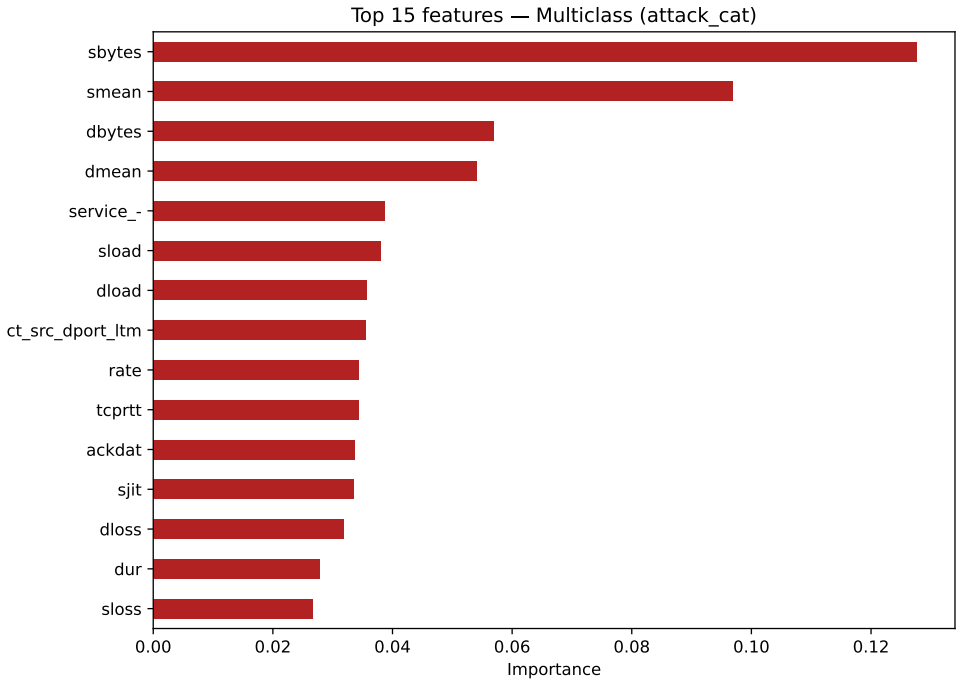


Figure: Multiclass Confusion Matrix

Feature Importance Analysis

- **Binary task**: Mostly relies on TCP handshake timing features (`ackdat`, `synack`, `tcprtt`), anomalous handshakes are a strong indicator of malicious traffic
- **Multiclass task**: Based on byte-transfer volume features (`sbytes`, `smean`, `dbytes`) to differentiate the *type* of attack





Project Evaluation

- **Performance & Operational Trade-offs**: The binary model achieves a strong F1-Score (**0.921**, 97% attack recall), but its **13% false positive rate** on normal traffic would produce a high volume of false alarms in real-world networks dominated by benign flows
- **The Multiclass Bottleneck**: Differentiating specific attack vectors remains challenging (**Macro F1: 0.561**), with concentrated overlap between related pairs (Backdoor/Analysis, DoS/Exploits) and between Fuzzers and Normal traffic (13%)
- **Robustness to Multicollinearity**: Manually dropping 10 highly correlated features ($r \geq 0.9$) slightly reduced CV Macro F1 (from **0.559** $\pm$ **0.001** to **0.552** $\pm$ **0.002**), confirming that tree-based models handle redundancy natively

Future Directions

To overcome the current limitations and push the system toward a realistic Network Intrusion Detection System, future iterations could explore:

- **Advanced Resampling**: Although SMOTE was initially considered, it was discarded due to extreme computational overhead. Future work must investigate highly optimized, scalable sampling techniques
- **Alternative Architectures**: Evaluating non-tree-based classifiers to determine if different mathematical boundaries can better isolate rare classes
- **Domain-Specific Feature Engineering**: Deriving new, highly specific network features aimed at better distinguishing the overlapping signatures of minority attacks from more frequent anomalies